Corso di Fondamenti di Informatica 2
CdL Ingegneria Informatica
Ing. Franco Zambonelli

ARCHITETTURA DEGLI ELABORATORI: ESERCIZI DOS/BIOS

Lucidi Realizzati in Collaborazione con:

Prof. Letizia Leonardi
Università di Modena

Prof. Antonio Corradi
Università di Bologna

ESERCIZIO: LETTURA DI UNA STRINGA (una serie di caratteri fino al carattere di INVIO) con funzioni DOS

```
COD     segment 'CODE'
        assume CS: COD, DS: DATI, SS: STACK1

main    proc    far
        push    ds
        mov     ax, 0
        push    ax
        mov     ax, DATI
        mov     ds, ax

        mov     bx, offset stringa

ciclo:  call    leggic
        cmp     al, CR    ; controllo sul carattere letto
        je      fine
        mov     byte ptr [bx], al
        inc     bx
        jmp     ciclo

fine:   ret
main    endp
leggic  proc    near

        mov     ah, 1
        int     21h ; DOS lettura carattere con echo
                    ; il risultato e' in AL
        ret
leggic  endp
COD     ends
```

```
DATI      segment 'DATA'
CR        EQU     0dh
stringa   db      120 dup(?)
DATI      ends

STACK1  segment stack 'STACK'
          db      256 dup('stack')
STACK1ends

end
```


ESERCIZIO: LETTURA DI UNA STRINGA (un numero prefissato di caratteri) con funzioni DOS

```
CODICE segment 'CODE'
assume CS: CODICE, DS: DATI, SS: STACK1

main      proc      far
          push      ds
          mov       ax, 0
          push      ax
          mov       ax, DATI
          mov       ds, ax
          mov       dx, offset stringa
          mov       ah, 0Ah
          int       21h ; DOS lettura di stringa con echo
                        ; il risultato e' in DS:DX
fine:     ret
main      endp

CODICE  ends

DATI      segment 'DATA'
stringa   db      11   ; numero di caratteri da leggere + 1
          db      ?    ; numero di caratteri letti
          db      10 dup(?)     ; BUFFER
DATI      ends

STACK1  segment stack 'STACK'
          db      256 dup('stack')
STACK1ends
          end
```

ESERCIZIO: SCRITTURA di una stringa con FUNZIONI DOS

```
CODICEsegment byte public 'CODE'
assume cs:CODICE, ds:DATI, ss: st1

main    proc    far
        push    ds
        mov     ax, 0
        push    ax

        mov     ax, DATI
        mov     ds, ax

        mov     dx, offset stringa
        mov     ah, 9
        int     21h ; DOS stampa di una stringa
    ; la stringa DEVE terminare con in carattere '$'

        ret
main    endp

CODICE ends

DATI    segment 'DATA'
stringa db  'Ciao','$'
DATI    ends

st1     segment stack 'stack'
        db  256 dup('stack')
st1     ends
        end
```


ESERCIZIO: definizione di una funzione, richiamabile da un programma C, che legge un carattere
===> USO del DOS e MODELLO SMALL

Programma C:

```
#include <stdio.h>
extern char readchar();     /* funzione Assembler */

void main()
{   char  c;
    c =readchar();
    printf("Il carattere letto %c\n", c);
}
```

Programma Assembler:

```
_TEXT   segment byte public 'CODE'
assume CS: _TEXT
public _readchar

_readchar  proc   near
        mov     ah, 1
        int     21h ; DOS lettura carattere con echo
                    ; il risultato in AL
        ret
_readchar   endp

_TEXT   ends
        end
```

ESERCIZIO: altra definizione di una funzione, richiamabile da un programma C, che legge un carattere C
===> USO del DOS e MODELLO SMALL

Programma Assembler:

```
_TEXT   segment byte public 'CODE'
assume CS: _TEXT
public _readchar

_readchar     proc       near
          mov       ah, 1
          int       21h ; DOS lettura carattere con echo
                        ; risultato in AL
          mov       cl, al    ; ATTENZIONE !!!
          mov       dl, LF  ; carattere da visualizzare
          mov       ah, 2
          int       21h     ; DOS scrittura su video
          mov       dl, CR ; carattere da visualizzare
          mov       ah, 2
          int       21h      ; DOS scrittura su video
          mov       al,  cl
          ; VALORE DI RITORNO IN AL ===> carattere letto
          ret
_readchar     endp
_TEXT   ends

LF        EQU     0ah     ; line-feed
CR        EQU     0dh     ; carriage-return
          end
```


ESERCIZIO: definizione di una funzione, richiamabile da un programma C, che legge una stringa C
===> USO del DOS e MODELLO SMALL

Programma C:

```
#include <stdio.h>
extern void readstring(char *s);      /* funzione Assembler */
void main()
{     char  s[120];
      readstring(s);
      printf("La stringa letta %s\n", s);
}
```

Programma Assembler:

```
_TEXT   segment byte public 'CODE'
assume CS: _TEXT
public _readstring

_readstring   proc       near
          push      bp
          mov       bp, sp
          push      si
          mov       si, [bp + 4]
ciclo:    call      leggic
          cmp       al, BLANK
          je        fine
          cmp       al, CR
          je        fine
          mov       byte ptr [si], al
          inc       si
          jmp       ciclo
fine:     call      acapo
```

```
        mov     al, 0; terminatore del C
        mov     byte ptr [si], al
        pop     si
        pop     bp
        ret
_readstring   endp

leggic  proc   near
        mov     ah, 1
        int     21h ; DOS lettura carattere
                    ; risultato in AL
        ret
leggic  endp

acapo   proc    near
        mov     dl, LF
        mov     ah, 2
        int     21h ; DOS scrittura su video
        mov     dl, CR
        mov     ah, 2
        int     21h ; DOS scrittura su video
        ret
acapo   endp
_TEXT   ends

LF      EQU     0ah
CR      EQU     0dh
BLANK   EQU     20h
        end
```


ESERCIZIO: definizione di una funzione, richiamabile da un programma C, che effettua la visualizzazione di una stringa C

===> USO del DOS e MODELLO SMALL

Programma C:

```
extern void writestring (char *s); /* funzione Assembler */
void main()
{     char  s[] = "Stringa da stampare";
      writestring(s);
}
```

Programma Assembler:

```
_TEXT   segment byte public 'CODE'
assume CS: _TEXT
public _writestring
_writestring  proc      near
              push      bp
              mov       bp, sp
              push      si
              mov       si, [bp + 4]
ciclo:        mov       dl, byte ptr [si]
              cmp       dl, 0; controllo sul terminatore del C
              je        fine
              call      stampac
              inc       si
              jmp       ciclo
fine:         call      acapo
              pop si
              pop bp
              ret
_writestring  endp
```

```
acapo   proc        near
        mov         dl, LF
        mov         ah, 2
        int         21h ; DOS scrittura su video
        mov         dl, CR
        mov         ah, 2
        int         21h ; DOS scrittura su video
        ret
acapo   endp

stampac  proc       near
        mov         ah, 2
        int         21h ; DOS scrittura carattere
; il carattere deve essere in DL
        ret
stampac  endp

_TEXT   ends

LF      EQU         0ah
CR      EQU         0dh

        end
```


ESERCIZIO: definizione di una funzione, richiamabile da un programma C, che effettua una letura della data
===> USO del DOS e MODELLO SMALL

Programma C:

```
#include <stdio.h>
extern void data(int *, int *, int *, int *); /* Assembler */

void main()
{  int anno, mese, giorno, gsett;
data(&anno, &mese, &giorno, &gsett);
printf("Anno %d\n", anno);
printf("Mese %d\n", mese);
printf("Giorno %d\n", giorno);
printf("Giorno della settimana %d\n", gsett);
}
```

Programma Assembler:

```
_TEXT   segment byte public 'CODE'
assume  cs: _TEXT
public _data

_data   proc    near
        push    bp
        mov     bp, sp
        push    si
        mov     ah, 2Ah
        int     21h         ; lettura della data
    ; in CX anno, in DH mese (da 1 a 12)
    ; in DL giorno (da 1 a 31)
    ; in AL giorno della settimana
    ; (0 domenica, 1 lunedi', etc)
```

```
        mov     si, [bp + 4]
        mov     word ptr [si], cx
        mov     bl, dh
        mov     bh, 0
        mov     si, [bp + 6]
        mov     word ptr [si], bx
        mov     si, [bp + 8]
        mov     bl, dl
        mov     word ptr [si], bx
        mov     bl, al
        mov     si, [bp + 10]
        mov     word ptr [si], bx
        pop     si
        pop     bp
        ret
_data   endp
_TEXT   ends


        end
```


ESERCIZIO: LETTURA di una stringa (una serie di caratteri fino al carattere di INVIO) con funzioni BIOS

```
CODICE      segment 'CODE'
assume CS: CODICE, DS: DATI, SS: STACK1

main    proc    far
        push    ds
        mov     ax, 0
        push    ax
        mov     ax, DATI
        mov     ds, ax

        mov     bx, offset stringa
ciclo:  call    leggic
        cmp     al, CR    ; controllo sul carattere letto
        je      fine
        mov     byte ptr [bx], al
        inc     bx
        jmp     ciclo
fine:   ret
main    endp

leggic  proc    near
        mov     ah, 0
        int     16h ; BIOS lettura carattere da  tastiera
                    ; il risultato e' in AL
        xor     ah, ah
        ret
leggic  endp
CODICE  ends
```

```
DATI        segment 'DATA'
CR          EQU     0dh
stringa     db      120 dup(?)
DATI        ends

STACK1      segment stack 'STACK'
            db      256 dup('stack')
STACK1      ends

            end
```


ESERCIZIO: LETTURA di una stringa con echo di ogni carattere con funzioni BIOS

```
CODICE segment 'CODE'
assume CS: CODICE, DS: DATI, SS: STACK1

main    proc    far

        push    ds
        mov     ax, 0
        push    ax
        mov     ax, DATI
        mov     ds, ax

        mov     si, offset stringa
ciclo:  call    leggic
        call    stampac ; ECO del carattere letto
        cmp     al, CR
        je      fine
        mov     byte ptr [si], al
        inc     si
        jmp ciclo
fine:   ret
main    endp

leggic  proc    near    ; uguale a prima
        mov     ah, 0
        int     16h ; BIOS lettura carattere
        xor     ah, ah
        ret
leggic  endp
```

```
stampac proc    near
        mov     ah, 9
        mov     bh, 0           ; numero di pagina video
        mov     bl, ATTR        ; attributo carattere
        mov     cx, 1           ; numero di ripetizioni
        int     10h ; BIOS scrittura carattere sul video
                    ; il carattere deve essere in AL
        ret
stampac endp

CODICE  ends

DATI        segment 'DATA'
CR          EQU     0dh
ATTR        EQU     07h
stringa     db      120 dup(?)
DATI        ends

STACK1      segment stack 'STACK'
            db      256 dup('stack')
STACK1      ends

            end
```


ESERCIZIO: LETTURA di una stringa con echo con funzioni BIOS

```
CODICE segment 'CODE'
assume CS: CODICE, DS: DATI, SS: STACK1

main    proc    far
        push    ds
        mov     ax, 0
        push    ax
        mov     ax, DATI
        mov     ds, ax
        mov     si, offset stringa
        call    pos ; legge posizione corrente del cursore
ciclo:  call    leggic
        call    stampac
        call    setpos  ; incrementa posizione cursore
        cmp     al, CR
        je      fine
        mov     byte ptr [si], al
        inc     si
        jmp     ciclo
fine:   ret
main    endp

pos     proc    near
        mov     ah, 3   ; legge posizione cursore
        mov     bh, 0   ; pagina video
        int     10h ; BIOS posizione cursore
                    ; DH nro riga e DL nro colonna
        ret
pos     endp
```

```
setpos  proc    near
        mov     ah, 2     ; cambia posizione cursore
        mov     bh, 0     ; pagina video
        inc     dl        ; incrementa la colonna
        int     10h ; BIOS posizione cursore
                          ; DH nro riga e DL nro colonna
        ret
setpos  endp
leggic  proc    near      ; uguale a prima
        mov     ah, 0
        int     16h ; BIOS lettura carattere
        xor     ah, ah
        ret
leggic  endp
stampac proc    near      ; uguale a prima
        mov     ah, 9
        mov     bh, 0
        mov     bl, ATTR
        mov     cx, 1
        int     10h ; BIOS scrittura carattere sul video
        ret
stampac endp
CODICE  ends
DATI    segment 'DATA'
CR      EQU     0dh
ATTR    EQU     07h
stringa db      120 dup(?)
DATI    ends
STACK1  segment stack 'STACK'
        db      256 dup('stack')
STACK1ends
end
```


ESERCIZIO: DEFINIZIONE di una funzione, richiamabile da un programma C, che legge un carattere
===> USO del BIOS e MODELLO SMALL

Programma C:

```
#include <stdio.h>
extern char readchar();  /* funzione Assembler */
main()
{     char  c;
      c = readchar();
      printf("\nIl carattere letto %c\n", c);
}
```

Programma ASSEMBLER:

```
_TEXT   segment byte public 'CODE'
assume CS: _TEXT
public _readchar

_readchar  proc   near
           call   pos
           call   leggic
           call   stampac
           call   setpos
           ret
_readchar  endp
pos        proc   near      ; uguale a prima
           mov    ah, 3
           mov    bh, 0
           int    10h ; BIOS posizione cursore
           ret
pos        endp
```

```
setpos  proc  near      ; uguale a prima
        mov     ah, 2
        mov     bh, 0
        inc     dl
        int     10h ; BIOS posizione cursore
        ret
setpos  endp

leggic  proc    near    ; uguale a prima
        mov     ah, 0
        int     16h ; BIOS lettura carattere
        xor     ah, ah
        ret
leggic  endp

stampac proc    near    ; uguale a prima
        mov     ah, 9
        mov     bh, 0
        mov     bl, ATTR
        mov     cx, 1
        int     10h ; BIOS scrittura carattere sul video
        ret
stampac endp
_TEXT   ends

ATTR    EQU     07h
        end
```


ALTRA DEFINIZIONE della FUNZIONE

Programma C:

```
#include <stdio.h>
extern char readchar();

main()
{     char  c;
      c = readchar();
      printf("Il carattere letto %c\n", c);
/*  in  questo caso la funzione readchar() deve  predisporre
il cursore a riga nuova */
}
```

Programma Assembler:

```
_TEXT   segment byte public 'CODE'
assume CS: _TEXT
public _readchar

_readchar  proc   near
           call       pos
           call            leggic
           call       stampac
           call       acapo
           ret
_readchar  endp
pos        proc     near      ; uguale a prima
           mov      ah, 3
           mov      bh, 0
           int      10h ; BIOS posizione cursore
           ret
pos        endp
```

```
acapo   proc    near
        mov     ah, 2
        mov     bh, 0
        inc     dh      ; si incrementa la riga
        mov     dl, 0   ; e si azzera la colonna
        int     10h ; BIOS posizione cursore
        ret
acapo   endp

leggic  proc    near    ; uguale a prima
        mov     ah, 0
        int     16h ; BIOS lettura carattere
        xor     ah, ah
        ret
leggic  endp

stampac proc    near    ; uguale a prima
        mov     ah, 9
        mov     bh, 0
        mov     bl, ATTR
        mov     cx, 1
        int     10h; BIOS scrittura carattere sul video
        ret
stampac endp
_TEXT   ends

ATTR    EQU     07h
        end
```


ESERCIZIO: definizione di una funzione, richiamabile da un programma C, che legge una stringa C
===> USO del BIOS e MODELLO SMALL

Programma C:

```
#include <stdio.h>
extern void readstring(char *s); /* funzione Assembler */

main()
{     char  s[120];
/* la stringa letta puo' essere al massimo di 119 caratteri */
      readstring(s);
      printf("La stringa letta %s\n", s);
}
```

Programma Assembler:

```
_TEXT   segment byte public 'CODE'
assume CS: _TEXT
public _readstring

_readstring   proc      near
        push    bp
        mov     bp, sp
        push    si

        mov     si, [bp + 4]    ; SI <- indirizzo inizio stringa
        call    pos
ciclo:  call    leggic
        cmp     al, BLANK; se il carattere e' lo spazio
        je      fine    ; si finisce
        cmp     al, CR  ; cosi' per il RITORNO
        je      fine
```

```
        mov     byte ptr [si], al
        call    stampac
        call    setpos
        inc     si
        jmp     ciclo
fine:   call    acapo
        mov     al, 0; si DEVE inserire il terminatore C
        mov     byte ptr [si], al

        pop     si
        pop     bp
        ret
_readstring   endp

pos     proc    near    ; uguale a prima
        mov     ah, 3
        mov     bh, 0
        int     10h ; BIOS posizione cursore
        ret
pos     endp

setpos  proc    near    ; uguale a prima
        mov     ah, 2
        mov     bh, 0
        inc     dl
        int     10h ; BIOS posizione cursore
        ret
setpos  endp
```



```
acapo   proc    near    ; uguale a prima
        mov     ah, 2
        mov     bh, 0
        inc     dh
        mov     dl, 0
        int     10h ; BIOS posizione cursore
        ret
acapo   endp

leggic  proc    near    ; uguale a prima
        mov     ah, 0
        int     16h ; BIOS lettura carattere
        xor     ah, ah
        ret
leggic  endp

stampac proc    near    ; uguale a prima
        mov     ah, 9
        mov     bh, 0
        mov     bl, ATTR
        mov     cx, 1
        int     10h; BIOS scrittura carattere sul video
        ret
stampac endp

_TEXT   ends

CR          EQU     0dh
BLANK       EQU     20h
ATTR        EQU     07h
        end
```

COMPITO

Scrivere una procedura ASSEMBLER richiamabile da un programma C che abbia la seguente interfaccia:

void leggi (char N, int D, char s[]);

La procedura leggi deve leggere da tastiera un insieme di caratteri. Il numero massimo di caratteri è dato dal parametro D: l'utente può inserire anche meno caratteri terminando l'immissione di carattere con il tasto di RITORNO. I caratteri letti vengono memorizzati nella stringa s se e solo se sono diversi dal carattere N, altrimenti sono scartati.

```
/* programma C */
#include <stdio.h>
extern void leggi(char N, int D, char s[]);
/* funzione ASSEMBLER */
main()
{     char  s[120],  N;  int  D;
      printf("Dammi il valore del carattere\n");
      scanf("%c", &N);
      printf("Dammi il valore del numero di caratteri\n");
      scanf("%d", &D);  leggi (N, D, s);
      printf("La stringa letta %s\n", s);
}
_TEXT   segment  byte public 'CODE'
          assume CS: _TEXT
          public _leggi
```



```
_leggi    proc      near
          push      bp
          mov       bp, sp
          push      si
          push      di
          mov       di, 0
; DI rappresenta il contatore dei caratteri
; effettivamente inseriti nella stringa
          mov       si, [bp + 8] ; carica l'offset della stringa s
ciclo:    call      leggic
          cmp       al, CR
          je        fine
          cmp       al, byte ptr [bp + 4] ; confronto con N
          je        ciclo                 ; si scarta il carattere
          mov       byte ptr [si], al
          inc       si    ; si incrementa posizione nella stringa
          inc       di    ; si incrementa il contatore dei
caratteri
          cmp       di, [bp + 6] ; controllo numero caratteri D
          je        fine
          jmp       ciclo

fine:     call      acapo
          mov       al, 0           ; terminatore del C
          mov       byte ptr [si], al
          pop       di
          pop       si
          pop       bp
          ret
_leggi    endp
```

```
acapo   proc    near
        mov     dl, LF
        mov     ah, 2
        int     21h     ; DOS scrittura su video
        mov     dl, CR
        mov     ah, 2
        int     21h     ; DOS scrittura su video
        ret
acapo   endp

leggic  proc    near
        mov     ah, 1
        int     21h     ; DOS lettura carattere
        ret
leggic  endp

_TEXT   ends
LF      EQU     0ah
CR      EQU     0dh
        end
```